Bhavya Dhanuka
CHRIST (Deemed to be University)

ASG Airlines: End-to-End Data Engineering

Problem Statement

ASG Airlines, a nationwide carrier, operates flights across multiple cities through its booking platforms, scheduling systems, and airport logs. The organization collects operational flight data from these various systems; however, inconsistencies in the dataset such as corrupted flight identifiers, inconsistent time formats, missing values, and unhandled overnight (cross-day) flight scenarios have created challenges in generating accurate operational insights.

Without a standardized and reliable data pipeline, the airline faces difficulties in calculating accurate flight durations, analyzing route-wise traffic, identifying delays and anomalies, and making data-driven operational decisions.

To improve operational efficiency and reporting accuracy, ASG Airlines aims to build an end-to-end data engineering pipeline that ingests, cleans, standardizes, and transforms flight data into a reliable analytical dataset for reporting and business intelligence purposes.

Solution

Summary

I implemented an end-to-end airline data engineering pipeline for the Neostats ASG Airlines use case. Ingested four datasets from an excel workbook flights, bookings, passengers, and payments and processes them using Python and Pandas. The pipeline performs schema validation, data profiling, standardization, data-quality checks, duplicate removal, transformation, referential-integrity validation, PII masking, KPI generation, and export of analytics ready CSV datasets.

Technology Stack

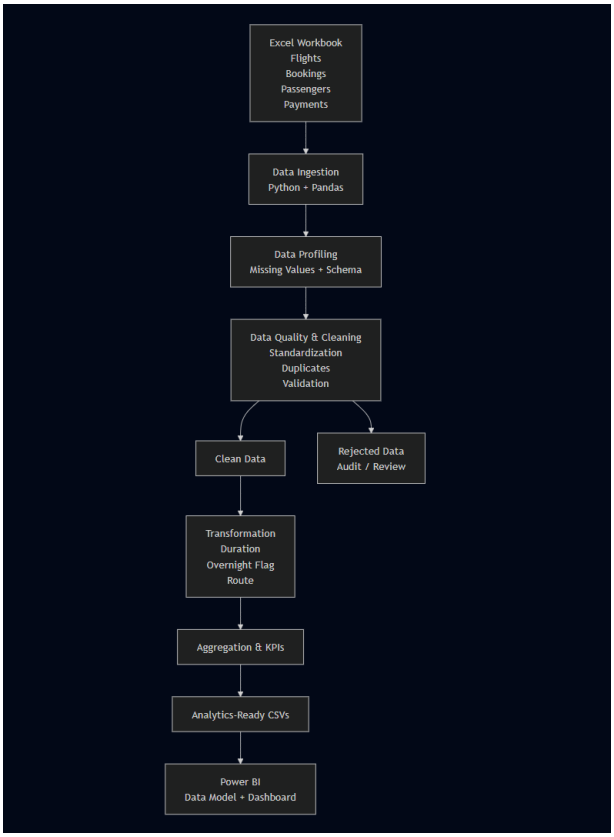
Source	Excel workbook (.xlsx)
Programming	Python 3.11
Data Processing	Pandas
Numerical Processing	NumPy
Output Format	CSV
Visualization / BI	Microsoft Power BI

Bhavya Dhanuka
CHRIST (Deemed to be University)

Dataset Structure

The source is one Excel workbook with four sheets, related through IDs:
1. flights - flight_id, airline, source, destination, departure_time, arrival_time, duration. Parent table.
2. bookings - booking_id, passenger_id, flight_id, status, booking_date, seat_number, passport_number, emergency_contact_name, emergency_contact_phone. flight_id references flights.
3. payments - payment_id, booking_id, payment_method, amount. booking_id references bookings.
4. passengers - passenger_id, first_name, last_name, email, phone, aadhaar_id, date_of_birth. Referenced by bookings.passenger_id.

Solution Architecture

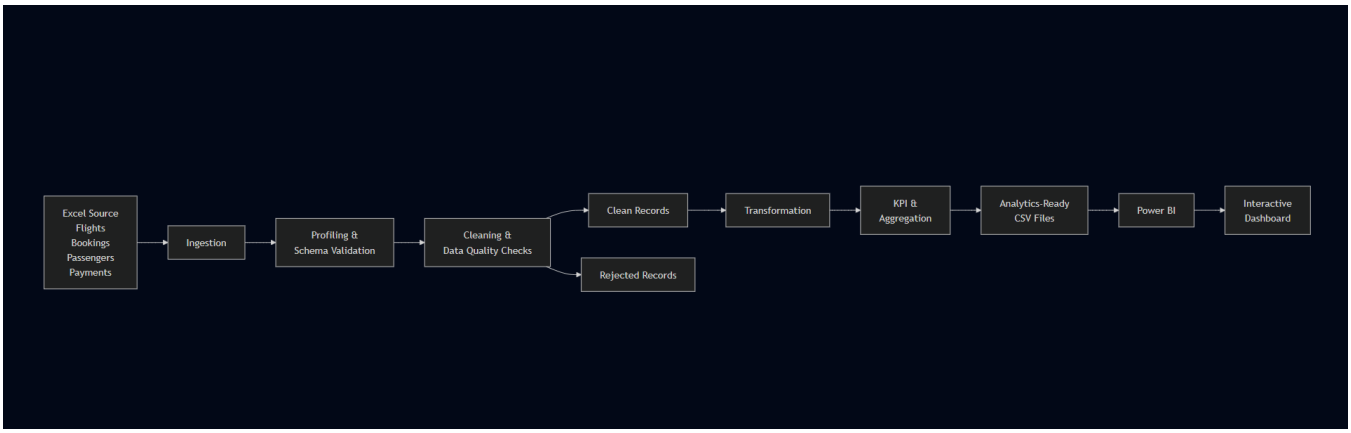


The solution follows:
Excel → Ingestion → Profiling → Validation/Cleaning → Transformation → Aggregation → CSV Outputs → Power BI
The architecture separates valid and rejected records so that invalid data is not silently discarded.
Loading: Validated and transformed datasets are exported as CSV files and loaded into Power BI for analytical reporting.
This is the local Python alternative permitted by the assignment, used instead of Azure Data

Bhavya Dhanuka
CHRIST (Deemed to be University)

Factory or Databricks because the dataset is small enough (approx 1000 rows) to process fully in memory with pandas.
Efficiency: Vectorized Pandas operations are used for validation, duplicate detection, filtering, transformation, and aggregation, which is appropriate for the current dataset size.
For actual scale, the same transformation logic can later be migrated to:
Azure Data Factory → Azure Data Lake → Databricks/Spark → Power BI
without changing the cleaning and transformation rules themselves only the execution engine changes.

Data Flow Diagram



Ingestion

The notebook reads each Excel sheet independently using Pandas. The source file can be supplied through the FLIGHT_DATA_FILE environment variable, with a default Excel filename used when the variable is not set.

Data Cleaning and Validation

Flight Data

- Required flight schema is validated.
- Column names and values are standardized.
- Flight identifiers are validated for missing values, and invalid records are rejected.
- Missing/unknown airlines are derived from known flight-ID prefixes.
- Departure and arrival timestamps are converted to datetime.
- Overnight flights are identified when arrival date is later than departure date.
- Duration is calculated from timestamps in minutes.
- Time, route and duration anomalies are flagged.
- Duplicate flights are detected using flight_id + departure_time.
- Clean and rejected flights are separated.

Bhavya Dhanuka
CHRIST (Deemed to be University)

Booking Data

- Booking fields are standardized.
- Booking dates are converted to datetime.
- Invalid/missing statuses are rejected.
- Duplicate booking_id values are checked.
- flight_id is validated against the clean flight dataset.
- Sensitive fields are masked.

Passenger Data

- Missing values are profiled.
- Duplicate passenger_id values are removed.
- Passenger records are validated.
- Names, email, phone and Aadhaar are masked.
- Date of birth is reduced to year.

Payment Data

- Payment fields are standardized.
- Amount is converted to numeric.
- Duplicate payment_id values are handled.
- Payment-to-booking referential integrity is checked.
- Invalid amounts and missing required fields are rejected.

Transformation Logic

Duration
Flight duration is calculated from:
arrival_time - departure_time
and stored as duration_minutes.

Overnight Handling
A flight is marked is_overnight = True when its arrival date is later than its departure date.

Route
A reporting route is created as:
source → destination
(This column is built as a calculated column in Power BI, not in the pandas pipeline.)

Referential Integrity
The following relationships are validated:

- Booking → Flight
- Payment → Booking

Bhavya Dhanuka
CHRIST (Deemed to be University)

Privacy & Access Control

Passenger PII was masked to demonstrate de-identification, while the flight analytics pipeline uses only the cleaned flight dataset. Sensitive passenger, booking, and payment data are not exposed in the Power BI analytical layer.

Masked fields include:

-Passenger name
-Email
-Phone
-Aadhaar
-Passenger ID
-Passport number
-Emergency contact details

Static masking was chosen over *hashing/tokenization* because the analytical layer doesn't need passenger identity or the ability to link a masked record back to the original person. Hashing or tokenization would preserve a consistent identifier that could support re-identification or joining that's not needed for the KPIs in this assignment, so it would just be extra exposure for no analytical benefit. Irreversible masking follows the principle of exposing only the minimum information required for analysis.

In a production Azure setup, access to the raw sensitive datasets would be restricted with role-based access control through Microsoft Entra ID. Only authorized users with a legitimate business need would get access to raw PII data engineers would have controlled access for pipeline processing, while analysts and Power BI users would only receive masked or aggregated datasets. The Power BI layer already reflects this in practice: it only reads masked/aggregated fields, so nothing sensitive is exposed at the report level even without the Azure RBAC layer in place. This follows the principle of least privilege sensitive data access limited to whoever actually needs it.

Analytics Outputs

The pipeline produces:

powerbi_flights.csv
clean_flights.csv
rejected_flights.csv
masked_passengers.csv
masked_bookings.csv
clean_payments.csv
rejected_payments.csv
kpi_summary.csv
route_traffic.csv
airline_distribution.csv
booking_status_distribution.csv
payment_method_distribution.csv
data_dictionary.csv

Bhavya Dhanuka
CHRIST (Deemed to be University)

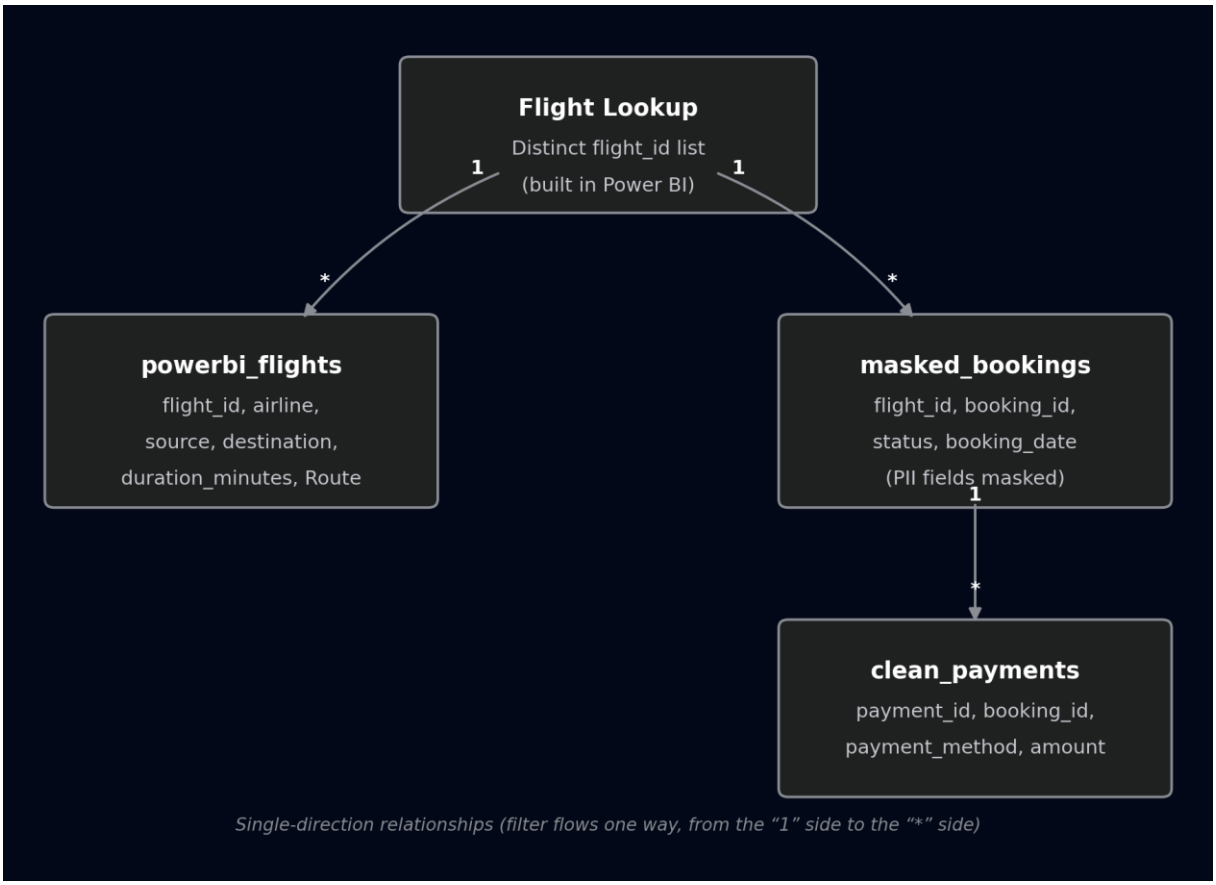
Power BI Data Model

A Flight Lookup table is used because flight_id is not unique in the flight dataset.

Relationships:

- Flight Lookup 1 —— * powerbi_flights (one to many)
- Flight Lookup 1 —— * masked_bookings (one to many)
- masked_bookings 1 —— * clean_payments (one to many)

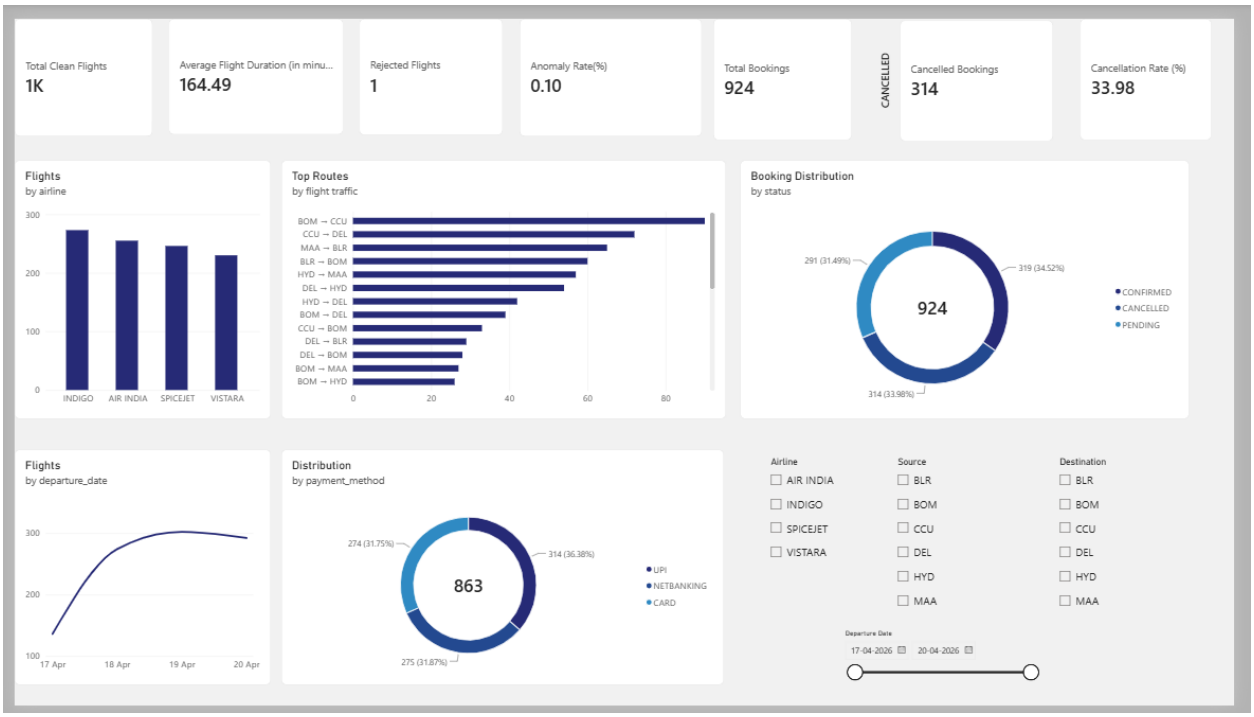
Single direction relationships are used.



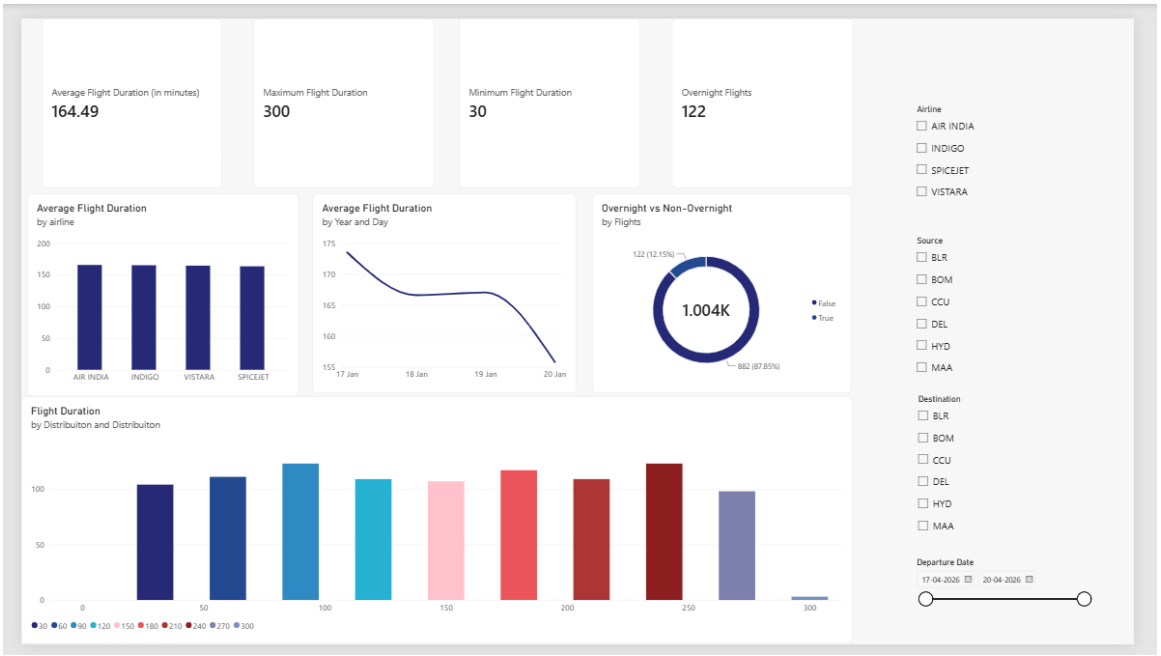
Bhavya Dhanuka
CHRIST (Deemed to be University)

Power BI Dashboard

1. **Overview**
High-level flight, booking and revenue KPIs with airline, route, date, booking and payment visuals.



2. Duration Analysis
Average, minimum and maximum duration, overnight flights and duration distributions.



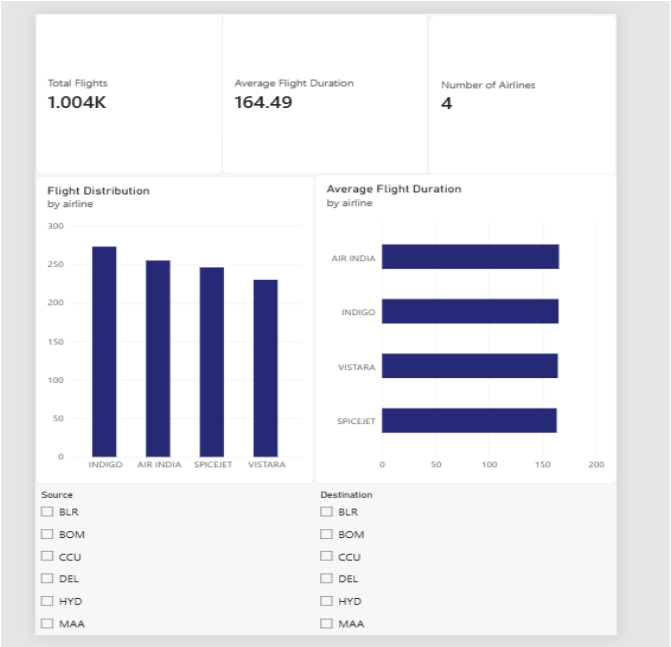
Bhavya Dhanuka
CHRIST (Deemed to be University)

3. Route Analysis

Route traffic, route count and average route duration.

4. Airline Analysis

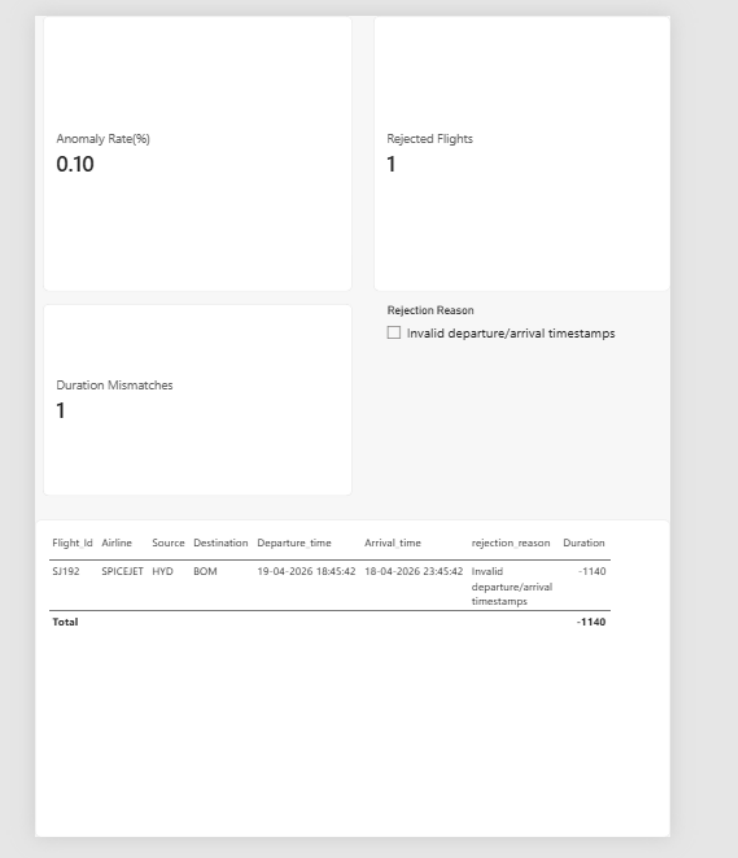
Flight volume and average duration by airline.



Bhavya Dhanuka
CHRIST (Deemed to be University)

5. Delay & Anomaly

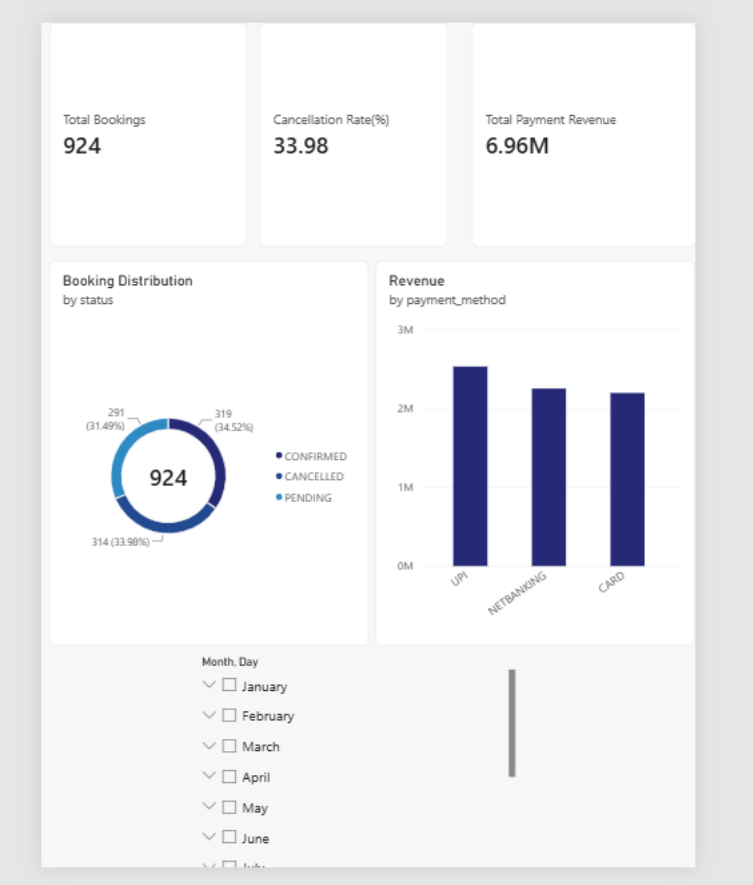
Rejected flight records, anomaly rate and duration mismatches.
Actual flight delay is not calculated because the source data does not contain scheduled-vs-actual timing fields.



Flight_Id	Airline	Source	Destination	Departure_time	Arrival_time	rejection_reason	Duration
SJ192	SPICEJET	HYD	BOM	19-04-2026 18:45:42	18-04-2026 23:45:42	Invalid departure/arrival timestamps	-1140
Total							**-1140**

Bhavya Dhanuka
CHRIST (Deemed to be University)

6. Booking & Revenue
Booking status, cancellation rate, revenue and payment-method analysis.



Key Business Insights

The solution supports operational decisions such as:
Identifying high-traffic routes.
Comparing airline flight volumes.
Monitoring flight-duration patterns.
Identifying overnight operations.
Monitoring booking cancellations.
Analyzing payment activity.
Detecting data-quality anomalies.

Assumptions

Airline can be derived from known flight-ID prefixes when missing.
Overnight means arrival occurs on a later calendar date.
Timestamp-derived duration is used for analytics.
Duplicate flights are identified using flight_id + departure_time.
Bookings must reference clean flights.

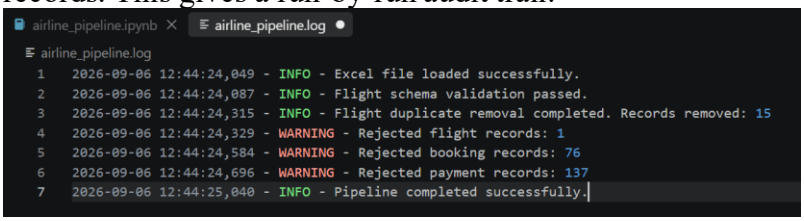
Payments must reference clean bookings.
PII is masked before reporting.
Delay cannot be reliably calculated from the supplied fields.

Error Handling and Validation

The notebook uses explicit validation checks and exceptions for critical failures, including schema errors, record-count mismatches, remaining invalid flight records and failed PII masking. Rejected records are retained separately for audit/review.

Logging

The pipeline logs each major stage (ingestion, schema validation, deduplication, rejection counts) with timestamps and severity levels INFO for successful steps, WARNING for rejected records. This gives a run-by-run audit trail.



Conclusion

The solution provides a complete flow from raw Excel data to validated, privacy-protected and Power BI-ready datasets:

Ingest → Validate → Clean → Transform → Protect → Aggregate → Model → Visualize